

Задача А. Вердикты системы All Cups

Сложим все вердикты, которые были получены на первом отборе, в какой-нибудь контейнер. Это может быть массив строк или `std::set<string>`. Затем пробежим циклом по множеству всех возможных вердиктов: OK, WA, PE, RE, ML и TL. Если очередной вердикт лежит в контейнере, то значит он уже был получен на первом отборе, а иначе его не было и можно сразу вывести этот вердикт на печать.

Задача В. N-факториал

N-факториалы растут очень быстро. Так, 5-факториал это $1^1 \cdot 2^2 \cdot 3^3 \cdot 4^4 \cdot 5^5 = 86400000$, а 6-факториал превышает 10^{12} . Так как сумма, заданная в задаче, ограничена числом 10^9 , то ее слагаемыми могут быть только N-факториалы для $N \leq 5$. Предподсчитаем все их значения.

Затем, также предподсчитаем все возможные суммы, которые могут получиться при сложении двух N-факториалов, за $O(N^2)$, и сохраним их в массиве.

Далее, получая число на вход, нам остается лишь проверять, присутствует ли оно в нашем массиве. Это можно сделать при помощи бинарного или простого линейного поиска.

Задача С. Граница

Представим карту страны как граф, в котором вершинам будут соответствовать города, а ребрам — дороги между ними.

Для каждой из столиц найдем кратчайшее расстояние от нее до любого города в стране. Это можно сделать, последовательно запустив алгоритм Дейкстры сначала из одной столицы, а затем из второй. Или можно запустить алгоритм Флойда, который находит все попарные расстояния в графе.

Далее проверим, существуют ли граничные города, то есть такие, расстояние до которых из обеих столиц одинаково. Если нет, то подходящего разбиения на княжества не существует (нельзя выполнить условие, что из одного княжества в другое можно попасть только через граничный город).

Если граничные города есть, удалим все ребра, ведущие в них. После этого граф должен распасться ровно на 2 компоненты связности. Все города в одной компоненте принадлежат одному княжеству.

При этом, столицы обязательно должны находиться в разных компонентах, и расстояние от любого города в компоненте до столицы его княжества должно быть строго меньше, чем расстояние до столицы другого княжества.

Если все вышесказанные условия выполняются, то разбиение на княжества возможно.

Задача D. Поход и озеро

Будем называть допустимыми те точки дороги, для которых выполнено свойство, что выйдя из этой точки в направлении перпендикулярном дороге Гена в какой-то момент дойдет до границы озера. Понятно, что допустимые точки дороги образуют отрезок — проекцию окружности озера на дорогу.

Пусть Гена вышел из допустимой точки и вдоль перпендикуляра дошел до озера, а потом от озера по прямой дошел до своего дома. Заметим, что либо этот маршрут будет корректным, либо он пересекает озеро еще в одной точке, через которую проходит более короткий корректный маршрут. Таким образом, мы можем взять в рассмотрение все допустимые точки, даже те которые порождают некорректные маршруты, и найти минимальный из таких маршрутов.

Функция расстояния в данной задаче устроена таким образом, что если перебрать точки из множества допустимых с достаточно маленьким шагом, то минимум из полученных маршрутов будет найден достаточно точно. То есть маленькая ошибка в выборе стартовой точки не приведет к большой ошибке в длине маршрута. Неформально можно сказать, что функция расстояния является «сжимающей».

Перебирать маршруты можно двумя способами. Способ первый: перебираем точки на нижней границе окружности (достаточно перебирать нижнюю-правую четверть окружности) и из этой точки опускаем перпендикуляр на дорогу, проводим отрезок к дому, а затем ищем сумму длин этих от-

резков. Способ второй: перебираем допустимые точки на дороге, потом ищем в какую точку окружности упрется перпендикуляр из этой точки, затем из точки окружности проводим отрезок к дому, и находим длину получившегося маршрута. Вывод формул остаётся в качестве упражнения читателю.

Задача Е. Горилла залезает на дерево

Решение 1

Давайте зафиксируем вершину v и решим задачу для неё.

Оценим количество рёбер, по которым горилла успеет подняться до того как устанет или доберётся до корня. К сожалению, таких рёбер может быть много. Покрасим рёбра веса 1 в красных цвет. Пусть не покрашенных рёбер p , тогда произведение весов на них хотя бы 2^p . Значит $p \leq \log_2 k \leq 30$.

То есть путь от вершины v до искомой будет состоять из путей, все рёбра которых имеют вес один, соединённых не более 30 рёбрами веса больше 1. То есть надо научиться подниматься из вершины в ближайшую вершину, на пути до которой встречается ребро веса хотя бы 2.

Несложно насчитать динамику, которая будет хранить ближайшего предка, на пути до которого встречается ребро веса хотя бы 2. Либо этот предок является нашим родителем (если вес ребра в него хотя бы 2), либо мы должны взять значение динамики от нашего родителя (если вес ребра в него 1).

Таким образом, выполнив предпосчёт за $O(n)$, можно независимо решить задачу для всех вершин за $O(n \log k)$.

Решение 2

Построим двоичные подъёмы за $O(n \log n)$. Будем решать задачу для фиксированной вершины прыжками по бинподъёмам. То есть будем искать момент, в который произведение на пути длины степень двойки станет больше k . Проверка на отсутствие переполнения при прыжке по бинподъёмам остаётся в качестве упражнения читателю.

Задача F. Название команды

Для каждого имени найдем все позиции его вхождения в название команды. Это можно сделать при помощи префикс-функции.

Так как количество имен небольшое, можем перебрать все варианты их перестановок, то есть рассмотреть все варианты порядка, в котором имена могут включаться в название команды.

Для каждой перестановки будем делать следующее:

1. найдем самое левое вхождение первого имени в перестановке;
2. найдем самое левое вхождение второго имени, такое, что оно начинается строго позже окончания первого;
3. повторим вышесказанное для всех последующих имен: для имени, которое должно входить в команду $(i+1)$ -м, будем искать самое левое вхождение, начинающееся строго позже найденного вхождения для i -го слова.

Если для каждого следующего имени удалось найти подходящее вхождение, то потенциальным ответом станет длина префикса, заканчивающегося вхождением $(n-1)$ -го слова.

Среди всех возможных ответов выберем минимальный. Если минимальная длина префикса равна m , то сократить название команды не получится и ответ будет -1 .